



TECHNISCHE
UNIVERSITÄT
WIEN

ottobock.

Framework for System Simulation

Thesis – Design and Implementation

Florian Frech, 03.11.2025

1. Overview Framework Classes

2. Core Abstractions

- CoSimComponent and Port System
- MultiComponent

3. Component Types

- FMUComponent
- OpenSImComponent
- FEMComponent

4. Concrete Models:

- OpenSimPendulum
- FEMPendulum
- MultiCompPendulum

5. System

- Class Diagram
- Execution Sequence Diagram

6. MultiCompPendulum

- Mode Switching and Synchronization Logic
- Model Comparison
- Test: Mode Switching and State Synchronization

7. Next Steps

Core Abstractions

- CoSimComponent:
 - Universal interface for all simulation components
 - Defines concrete and abstract lifecycle and interface methods
- MultiComponent
 - Enables heterogeneous simulation by switching between multiple models at runtime

System Integration

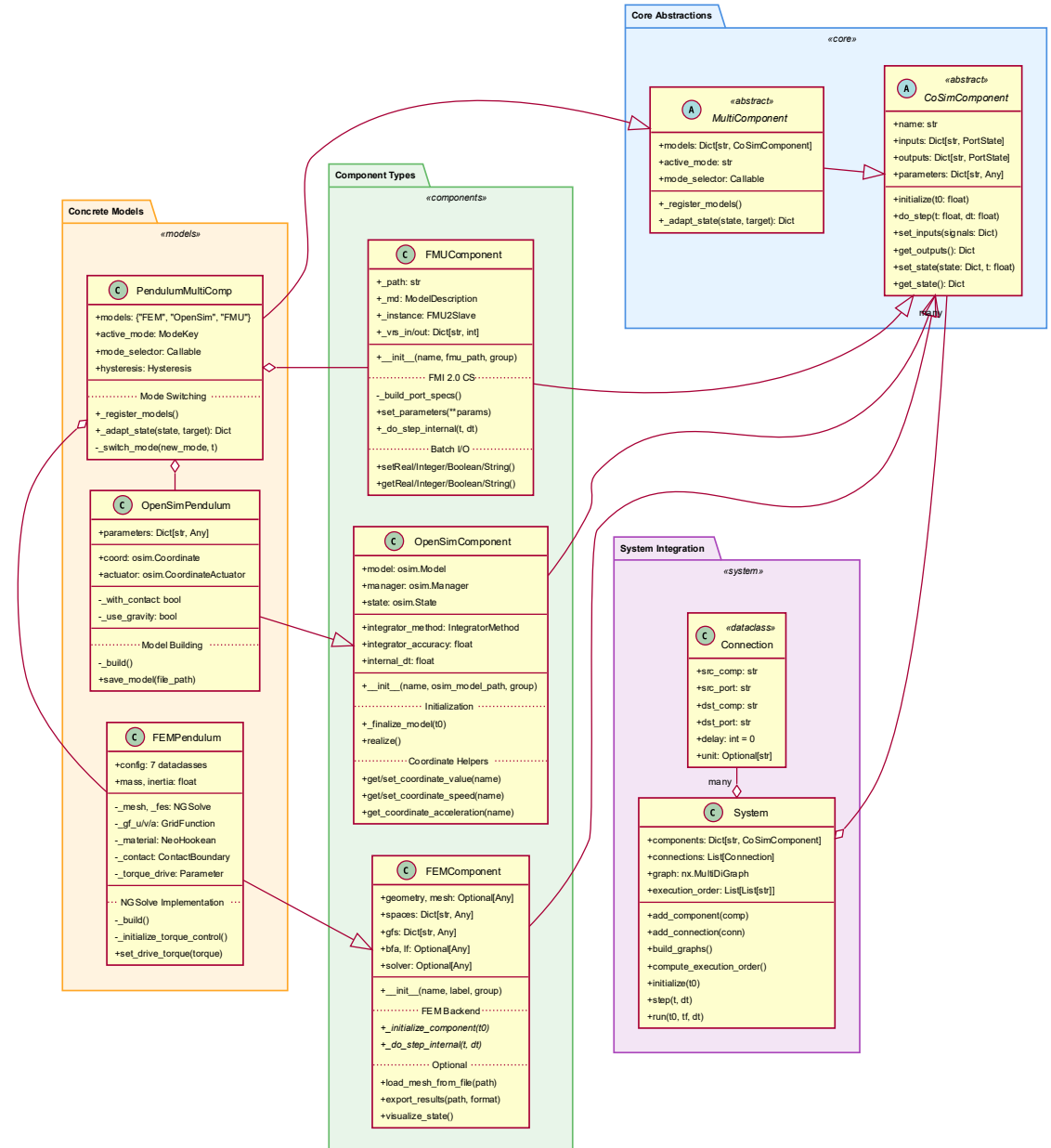
- Connection: declares signal routing between CoSimComponents
- System: Manages component lifecycle, signal routing, and execution order

Component Types

- FMUComponent (FMI 2.0 CS)
 - Integrates equation-based models from Modelica or other tools via FMU standard
 - Identifies inputs, outputs, and parameters by analyzing FMU's model description file
- OpenSimComponent
 - Wraps OpenSim Model for biomechanical simulation, musculoskeletal modeling, and multibody dynamics
- FEMComponent
 - Abstract FEM base class (backend-independent interface = no NGSolve imports)
 - Attributes: Geometry & Mesh, FE Space, Grid Functions, Visualization scene

Concrete Models - Controlled Pendulum

- PendulumMultiComp:
 - Combines OpenSim, FMU, and FEM pendulum models within a single CoSimulationComponent
 - Manages state switching and model synchronization
- OpenSimPendulum: User-defined OpenSim pendulum implementation
- FEMPendulum: User-defined netgen/NGSolve pendulum implementation



PortSpec: Immutable Port Specification

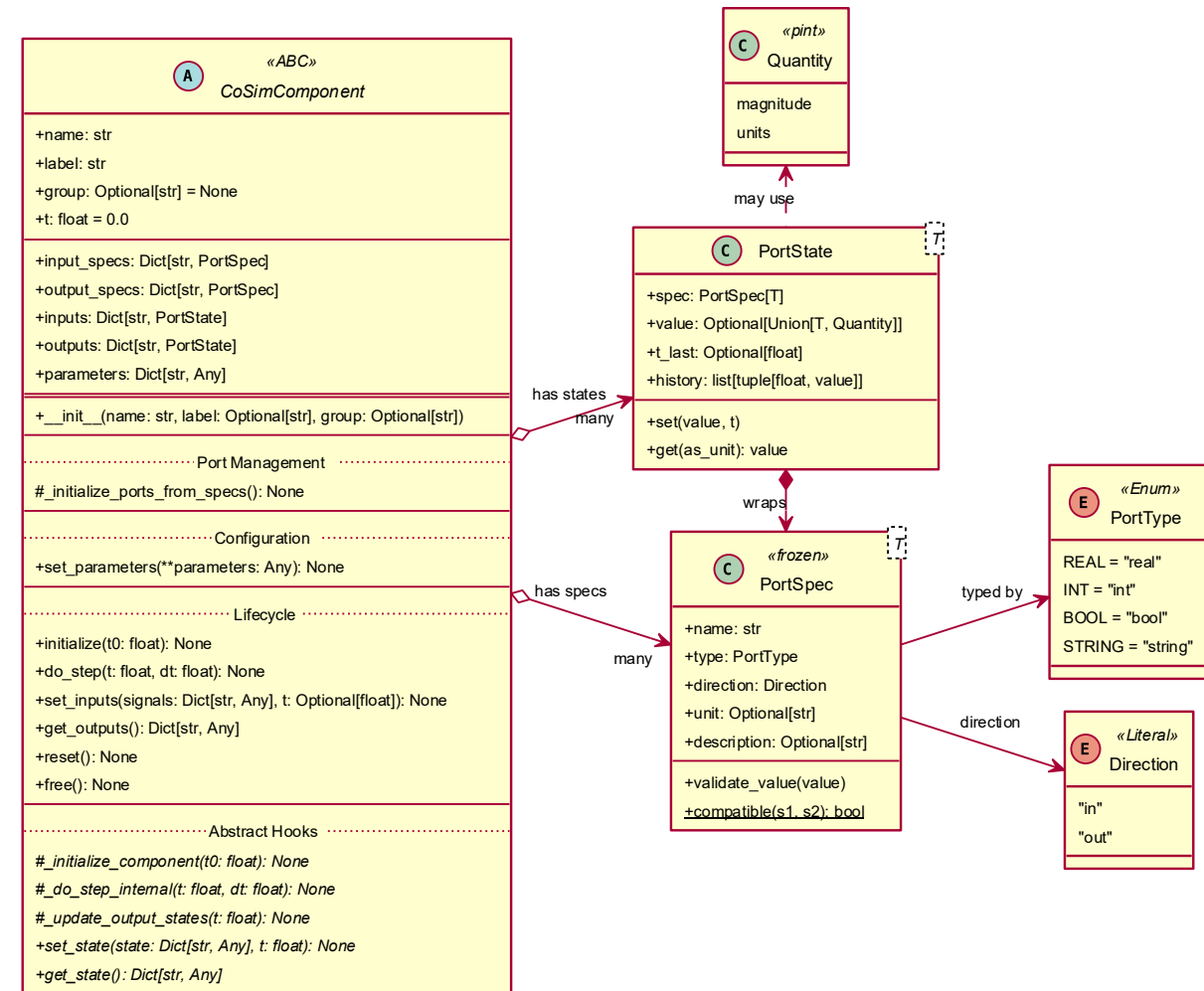
- Immutable specification of a component port which defines the interface
- Generic type T can be float, int, bool, str
- Direction: Input ('in') and Output ('out')
- Optional Unit for REAL Port Type (dependency on pint for unit handling)
- Ensures type safety and validation of port compatibility

PortState: Mutable Port Runtime State

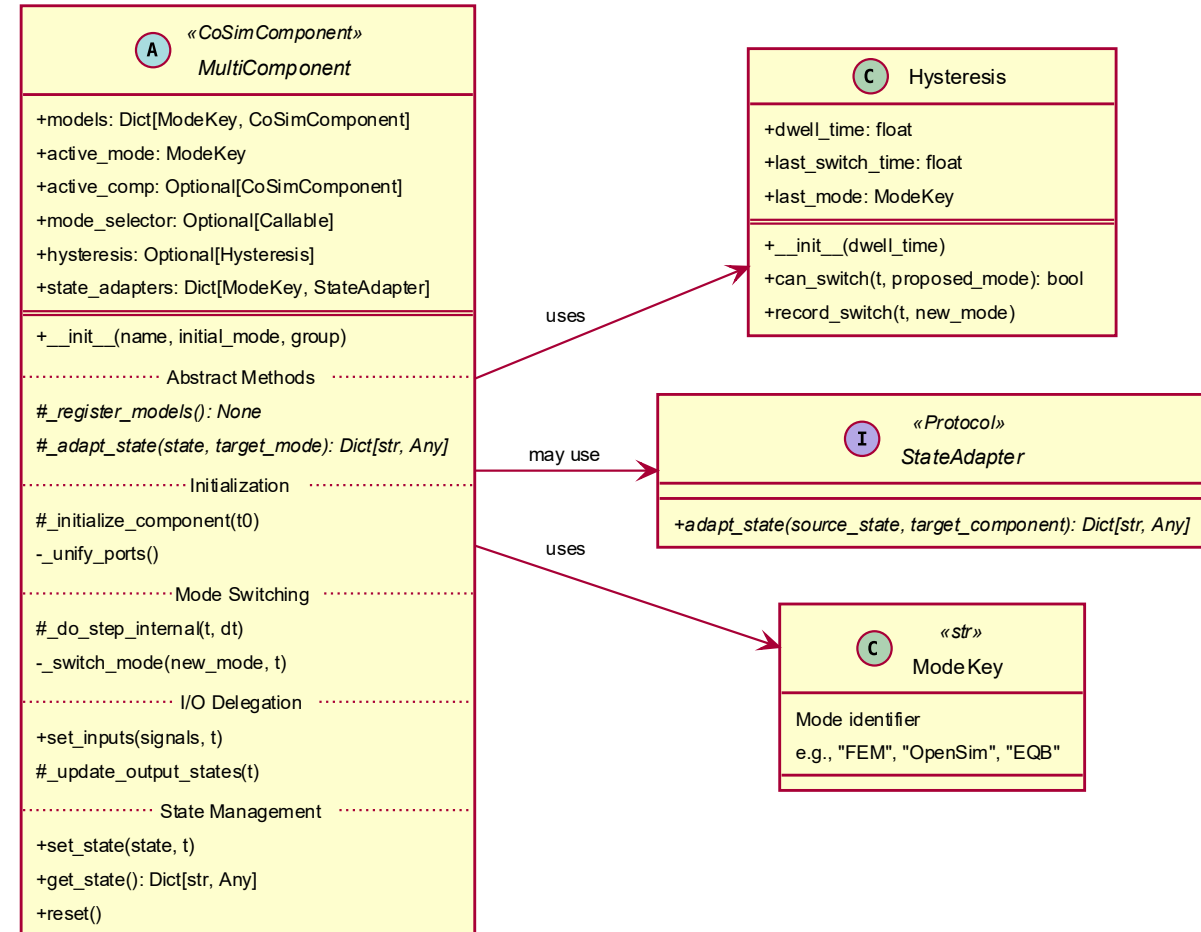
- Updated during simulation steps
- Tracks the dynamic state of the port (value and time of the last update)
- Maintains a complete history record of (time, value) pairs

CoSimComponent:

- Abstract base class provides interface for FMU, FEM, and OpenSim, Multi, and user-defined simulation components
- PortSpec Class is used to define the interface contract between connected CoSimComponents
- Mutable port state tracks the current value, time and history
- **Template Methods:**
 - initialize() and do_step() call the subclass methods _initialize_component() and _do_step_internal()
 - both calls _subclass_update_output_states to ensure ports are updated
- **Abstract Methods:** _initialize_component(), _do_step_internal(), _update_output_states(), state setting and getting, resetting
- **Concrete Methods:** ports initialization, setting of parameters and inputs

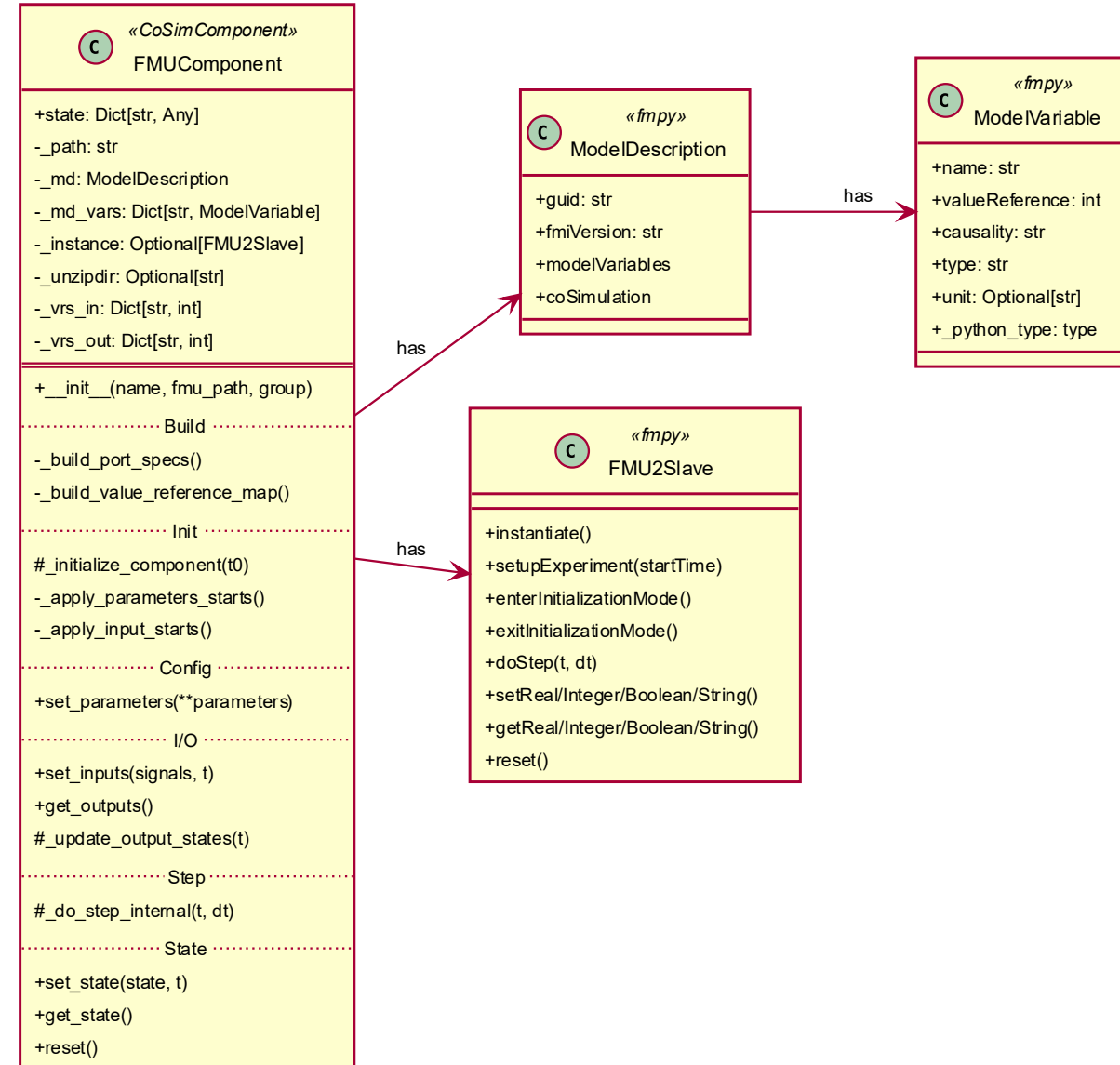


- Abstract composite component that enables runtime model switching between multiple heterogeneous simulation models (e.g., FEM, multibody, FMU) while maintaining a unified interface
- MultiComponent treats multiple sub-components uniformly via the CoSimComponent Interface
- Pluggable mode selector allows custom switching logic
- The base class orchestrates lifecycle, subclass implement specific methods
- **Model Registry:**
 - Registry of alternative implementations of the same physical system
 - ModeKey is used as model identifier
- **Active component:**
 - There is always only one of the models active for performing a simulation step
- **Switching Strategy:**
 - Function that decides when to switch modes based on time/state
- **Hysteresis:**
 - *dwell_time*: Minimum time a mode must be active to prevent mode chattering
 - Tracks last switch time and last mode and allows or prevents mode switching
- **StateAdapter Protocol:**
 - Optional interface for complex state transformations
- **Port Compatibility:**
 - All models must have identical port names
 - Compatible types and consistent units (e.g., all angles in rad)
 - *_unify_ports()* ensures that all involved CoSimComponents are compatible

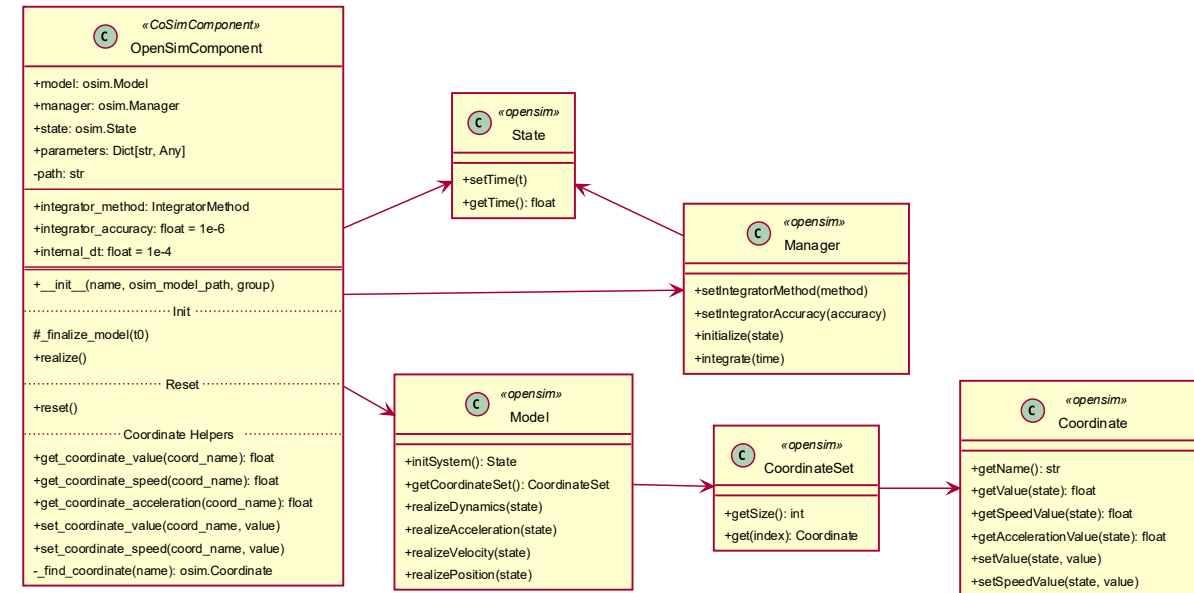


3. Component Types: FMUComponent

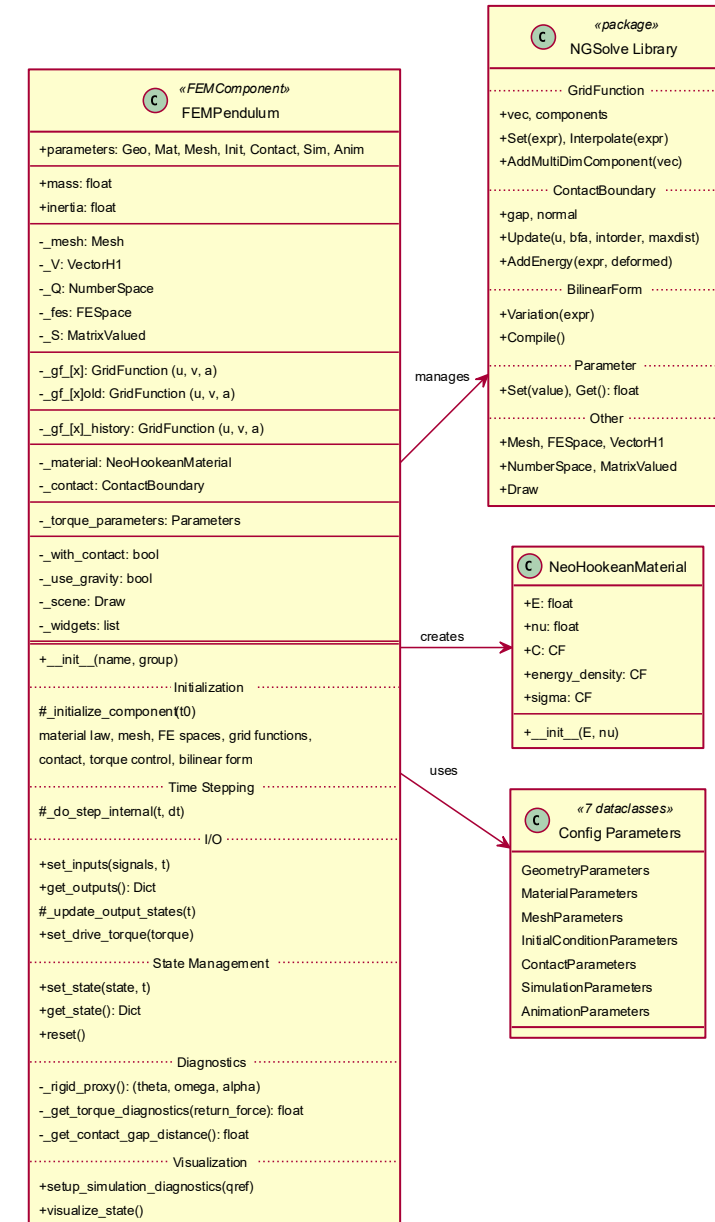
- Implements an FMU wrapper that integrates FMI Co-Simulation models into the co-simulation framework (bridge between standardized CoSimComponent and actual FMU instance)
- Loads and validates FMU files and extracts metadata for port and parameter setup
- **Automatic Port Discovery:**
 - Automatically creates port specification from FMU model description
 - Maps FMU variables to typed ports and preserves unit information
- **Value Reference Caching:**
 - Pre-computes FMU value references for efficient batch operations
 - No need for runtime lookups in model description during simulation
- **Batch I/O operations:**
 - Groups inputs data type for efficient FMU API calls
 - FMU API calls are only called once per type
 - Handles unit conversion for REAL ports (only magnitude of pint.quantity is set)
 - Validates input types against port specification
- **Lifecycle Management:**
 - Follows FMI initialization protocol: create instance, instantiate, setup experiment time, enter initialization mode, set parameters, and exit initialization mode
 - Sets up port states and caches for runtime efficiency
- **Simulation Stepping:**
 - Advances FMU simulation by one communication step
 - Updates output port states with new values
- **Unit-Aware Output Handling:**
 - Retrieves output in batches by type
 - Automatically attaches units to REAL quantities
 - Updates port states with time stamps



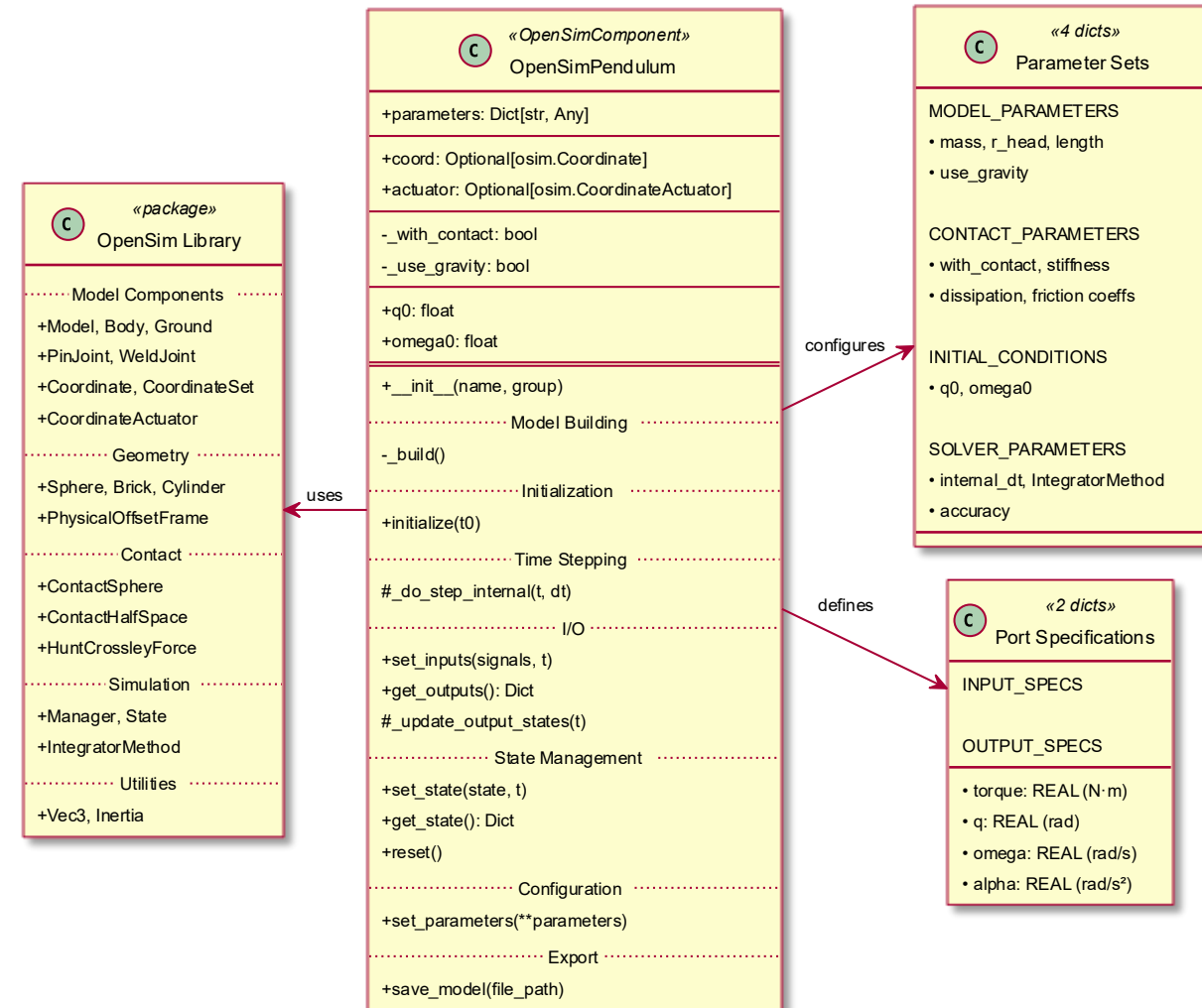
- Base class for OpenSim-based multibody dynamics component in the framework
- Implements CoSimComponent interface for OpenSim components by wrapping OpenSim's C++ API via Python bindings
- **Core OpenSim Integration:**
 - Model: OpenSim musculoskeletal/mechanical model
 - State: Current simulation state (positions, velocities, etc.)
 - Manager: Time integration manager (with configurable integrator method, accuracy, and internal time step)
- **Initialization Workflow:**
 1. Subclass builds model with geometry, bodies, joints, forces, etc.
 2. Base class finalizes via `_finalize_model(t0)`: initializes the system state; realizes computation of position, velocity, dynamics and acceleration; configures and initializes the manager
- **Coordinate Access:**
 - Via unified interface for generalized coordinates (e.g., joint angles)
 - using name-based access for getting and setting coordinate value, speed, and acceleration
- **State Management:**
 - Encapsulates all DOFs, time, and constraints
 - State is maintained across `do_step()` calls
- **Time Integration:**
 - Enables internal sub-steps between co-simulation steps with `_internal_dt`
 - Error control: Based on `integrator_accuracy` parameter



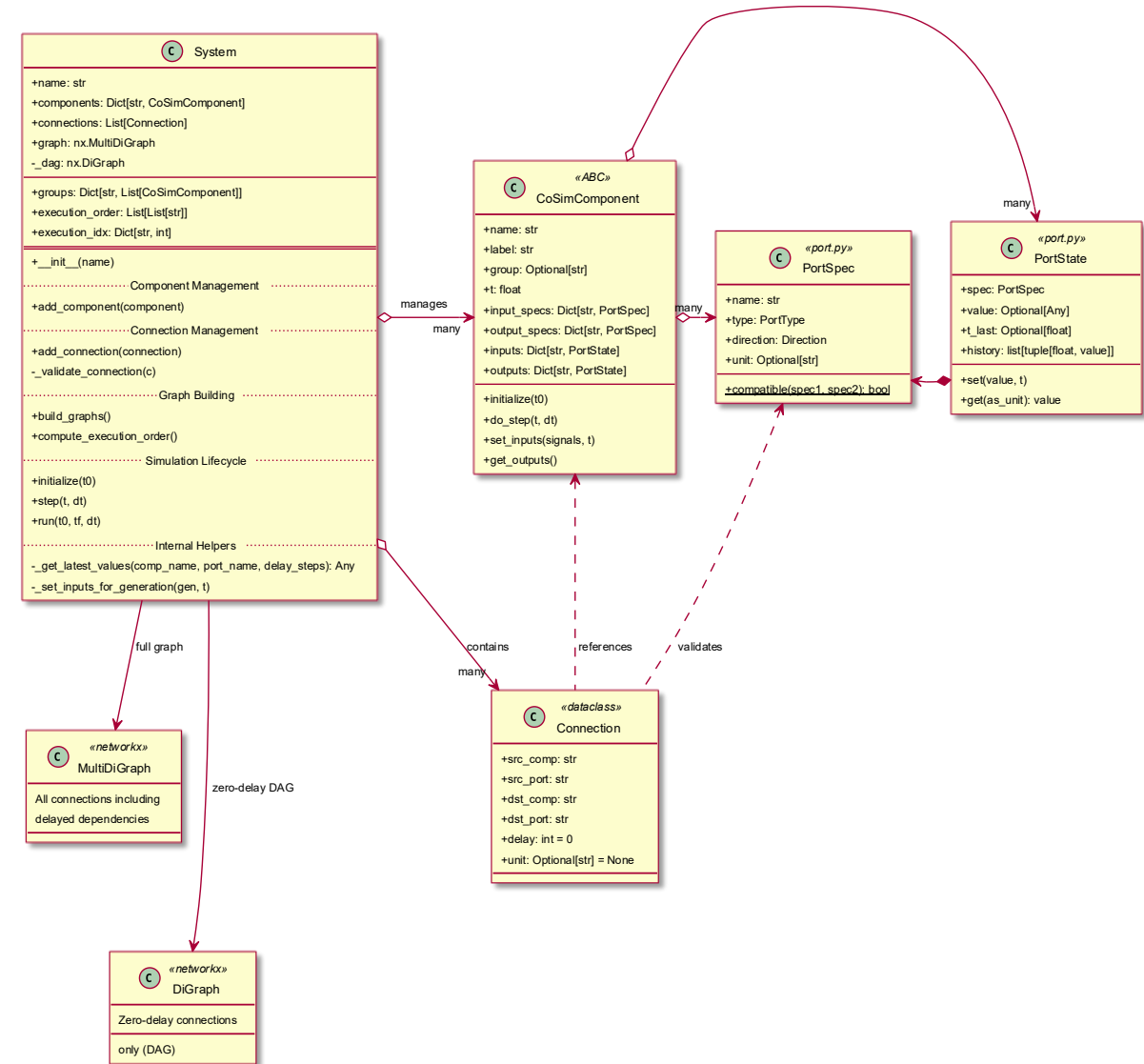
- Extends the FEMComponent base class and implements co-simulation netgen/NGSolve pendulum with torque input and rigid proxies for angular position, velocity, and acceleration as output
- **Configuration:** Modular parameter classes defined in pendulum_config.py allow flexible configuration for geometry, material, mesh, simulation, contact, and animation of outputs
- **Material model:** based on Neo-Hookean hyper-elasticity for large deformations (energy-based formulation)
- Distributed **traction-based torque application** on rotation edge in **current configuration**
- **Contact Mechanics** implemented using incremental gap formulation between pendulum head and wall with penalty stiffness
- **Adaptive time-stepping** based on gap distance to ensure energy conservation
- **Newton minimization** with sparse Cholesky factorization for solving for the displacement field
- **Newmark time integration** (implicit) for velocity and acceleration updates
- **Rigid Body Proxy:** extracts lumped parameters for angular position, velocity, and acceleration from continuum solution to enable seamless mode switching with rigid body models
- **State Management:**
 - Setting state converts angular position and velocity to displacement fields via rotation matrix
 - Getting state extracts the rigid body weight via weighted integrals and returns current torque from distributed traction diagnostics
- **Diagnostics & Visualization:**
 - Real-time widgets for time, time step, simulation mode
 - Reference position and pendulum state variable values
 - Applied torque and contact gap distance
 - Deformed mesh visualization with stress coloring



- Extends the OpenSimComponent base class and implements a single DOF pendulum
- **Model Structure:**
 - Fix *WeldJoint* for Ground to Base / *PinJoint* for Base to Head
 - Point mass head with spherical geometry
 - Coordinate actuator for direct torque application
- **Key Components:**
 - *coord*: Single rotational coordinate (q)
 - *actuator*: Coordinate actuator acting on q for torque input
 - *manager*: OpenSim manager for time integration
 - *state*: System state tracking configuration (position, velocities, accelerations, etc.)
- **Model Configuration:**
 - Model parameters: mass, length, head radius, *use_gravity*
 - Contact parameter: *with_contact*, stiffness, dissipation, friction coefficients for Hunt-Crossley Contact Force
 - Initial conditions: angular position and velocity
 - Solver parameters: internal time step, integrator method, accuracy
- **Physics Features:**
 - Rigid body dynamics with optional gravity (validation purpose)
 - Optional Hunt-Crossley contact (wall collision at $q=0$)
 - Direct torque actuation with override control
- **State Management:**
 - *set_state()*: Reinitializes system with new q, omega, and torque
 - *get_state()*: Returns ang. position, velocity, acceleration, and torque
- **Integration:**
 - Variable internal time step (*internal_dt* = $1e-4$)
 - Runge-Kutta-Merson integrator (explicit, 4th/5th order with adaptive step size)
 - Stage realization before integration (get new acceleration from new torque)



- System acts as the **co-simulation orchestrator** that manages interconnected components, validates signal flow, resolves computational dependencies, and executes time-stepping
- Component Management**
 - Maintains registry of all simulation components and their interconnections
 - Optional grouping for organization
- Connection Validation**
 - Components must exist before connection
 - Ports must be correctly typed (from output port to input port)
 - Type and unit compatibility between ports must be ensured
 - No duplicate connections or multiple feeding allowed
- Graph-Based Dependency Management**
 - Full graph: complete system topology including delayed connection
 - Directed Acyclic Graph (DAG): Only zero-delay connections for execution ordering
- Execution Order Computation**
 - Builds DAG only with zero-delay connections and checks for algebraic loops
 - Uses topological generation sort algorithm on zero delay DAG to get the components grouped into parallel generations
 - Topological ordering: ensures the dependencies are satisfied before execution
 - Parallel Generations: Components in the same generation can execute concurrently
- Simulation Execution:**
 - Phase 1: Set inputs for all generations**
 - Phase 2: Time integration for all generations**
 - Separating in two phases ensures consistent causality within each generation
 - All components see the same state of the system
 - Components in same generation can run concurrently
 - Delayed value retrieval enables by port history
 - Multiple drivers for single port are detected



Initialization Phase:

1. Component Setup

- Add components to system registry and group components by functional role (for system graph visualization)
- Configure parameters before initialization

2. Connection Definition

- Define signal routes between components (Connection objects)
- Validate ports, detect duplicates, and set delays

3. Graph Construction

- Full Graph: All connections (MultiGraph) for visualization
- Computational Directed-Acyclic Graph with zero-delay connections (DiGraph)
- Cycle Detection: Algebraic loops rejected unless broken by delays

4. Execution Order

- Topological sort: group components into parallel generations

5. Component Initialization

- Call `_initialize_component(t0)` for each component & write initial outputs

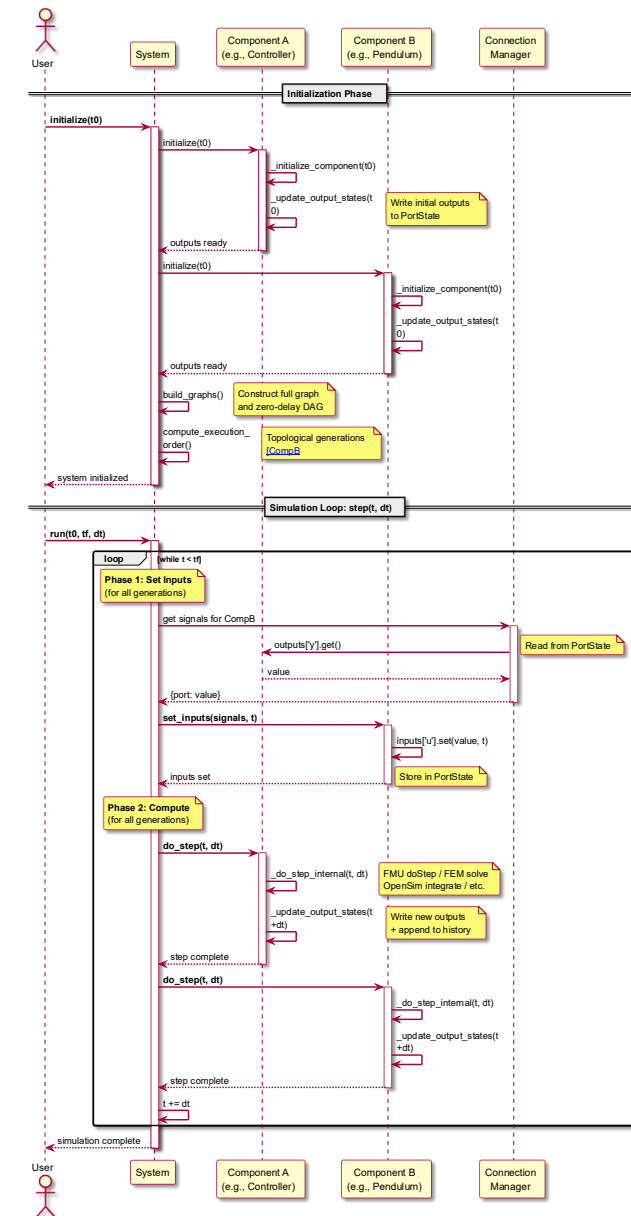
Execution Phase:

Phase 1: Set inputs for all generations sequentially

1. Route signals: for each component, gather incoming connections
2. Handle delays: read current / history value
3. Apply: call `component.set_inputs(signals, t)` to store input in PortState

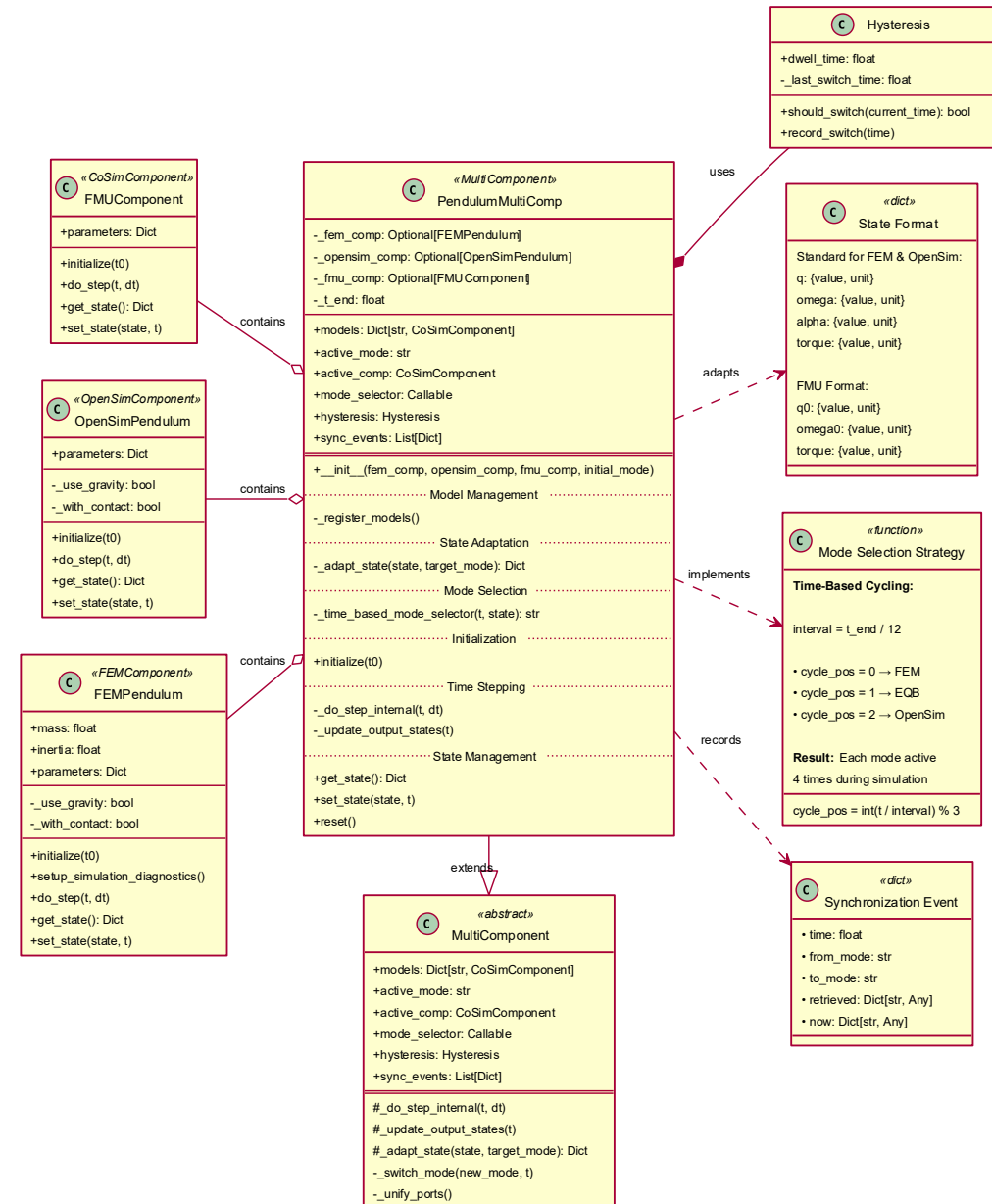
Phase 2: Compute for all generations sequentially

1. Execute: Call `component.do_step(t, dt)` for each component (executes internal step, subclass implementation)
2. Update outputs: Writes new values and appends to history (in PortState)
3. Advance time: $t += dt$



6. MultiCompPendulum: Class Diagram

- Implements a heterogeneous pendulum model consisting of different pendulum representations
 - FEM: Continuum mechanics with deformable body dynamics
 - OpenSim: Rigid body pendulum
 - EQB: Equation-based Modelica FMU
- **Mode Switching:**
 - Time-based cyclic strategy: each model is 4 x active during simulation
 - Hysteresis mechanism to prevent mode chattering
 - Automatic state synchronization at mode transitions
- **Parameter Synchronization:**
 - FEM serves as parameter master for mass, inertia, and geometry
 - Automated derivation of equivalent parameters for OpenSim and FMU
 - Consistent initial condition across all models
- **State Format Adaption:**
 - Used for the translation between component-specific interfaces
 - Standard: {q, omega, alpha, torque}
 - FMU: {q0, omega0, alpha, torque} (initial parameters are required for FMU reinitialization)
- **Validation & Diagnostics:**
 - Synchronization event tracking (for before/after state comparison)
 - Records: timestamp, mode transition state differences
 - Enables numerical accuracy analysis across all models
- **Unified Co-Sim Interface:**
 - Single component abstraction for multi-model system
 - Standardized ports: torque input; angular position, velocity and acceleration as outputs
 - Drop-in replacement for single model-component

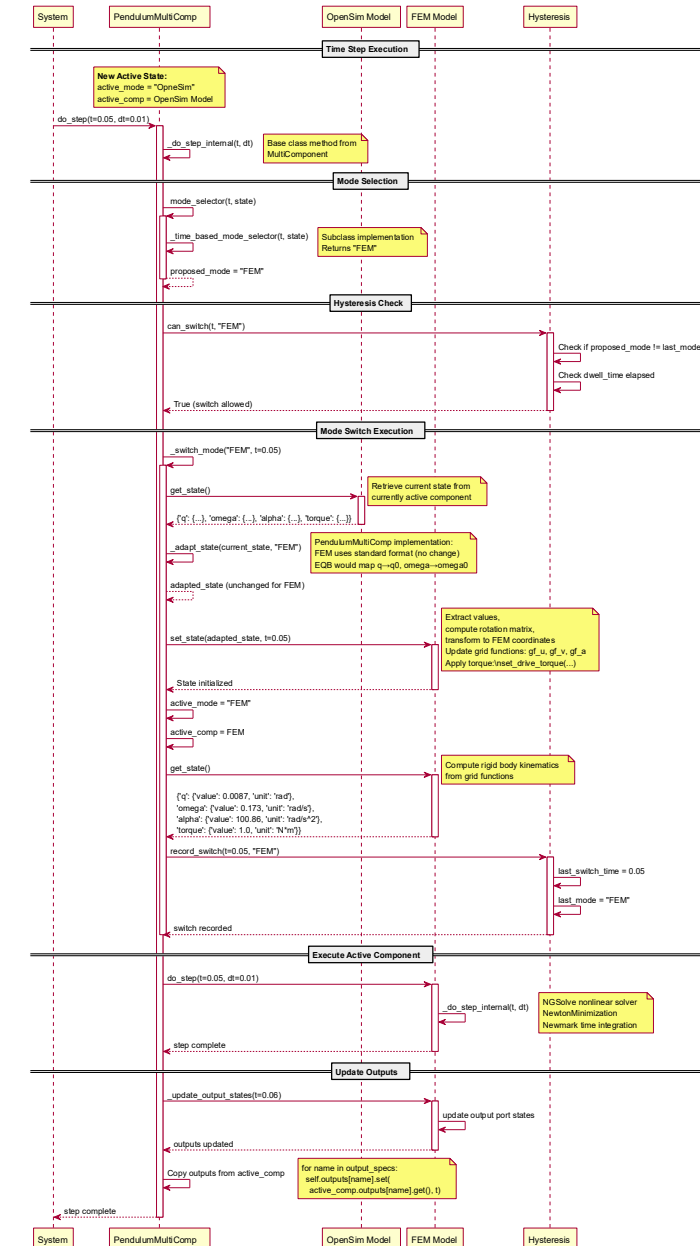


State Transfer: Extraction / getState()

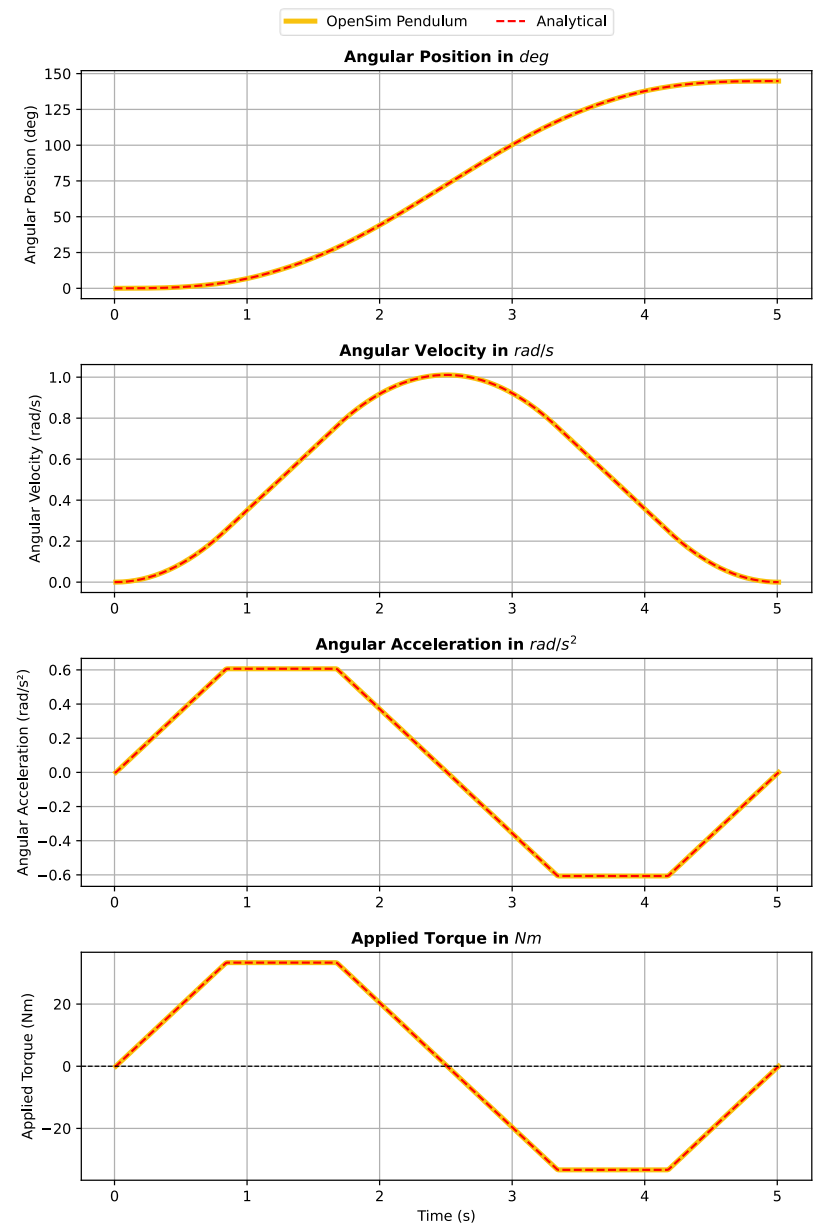
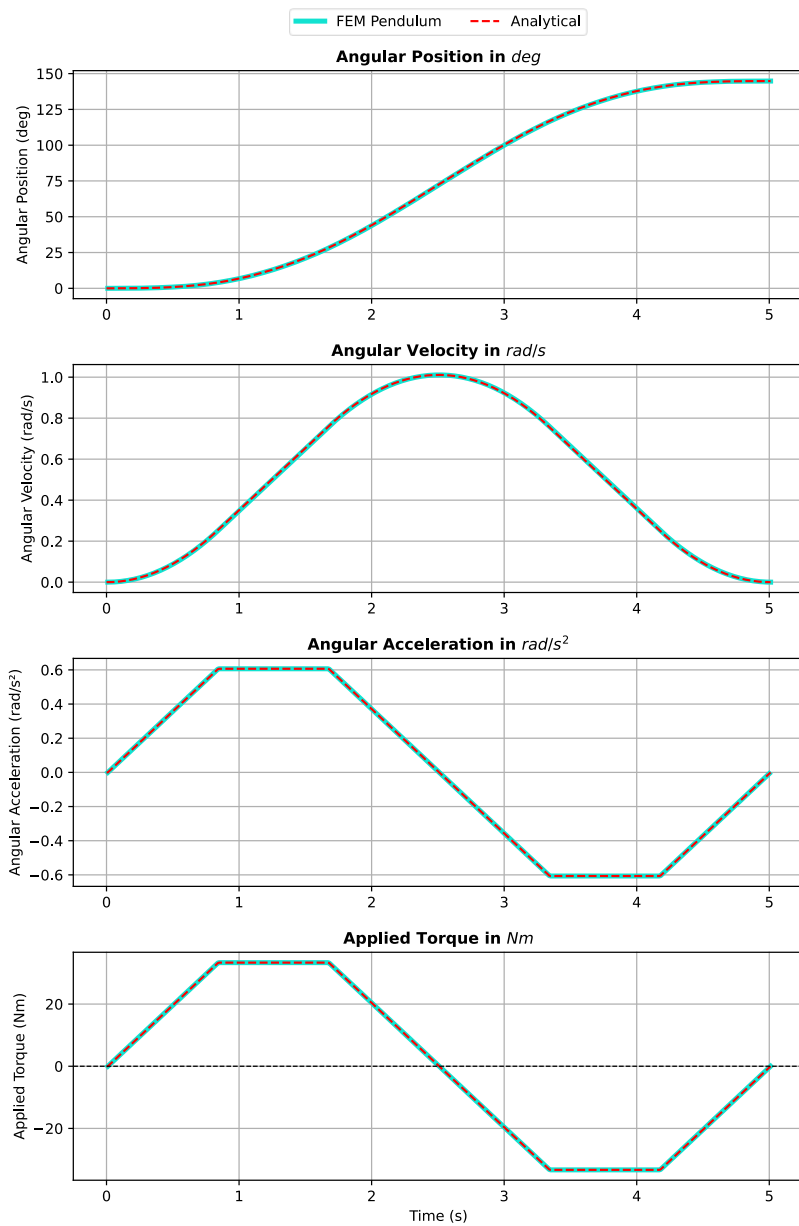
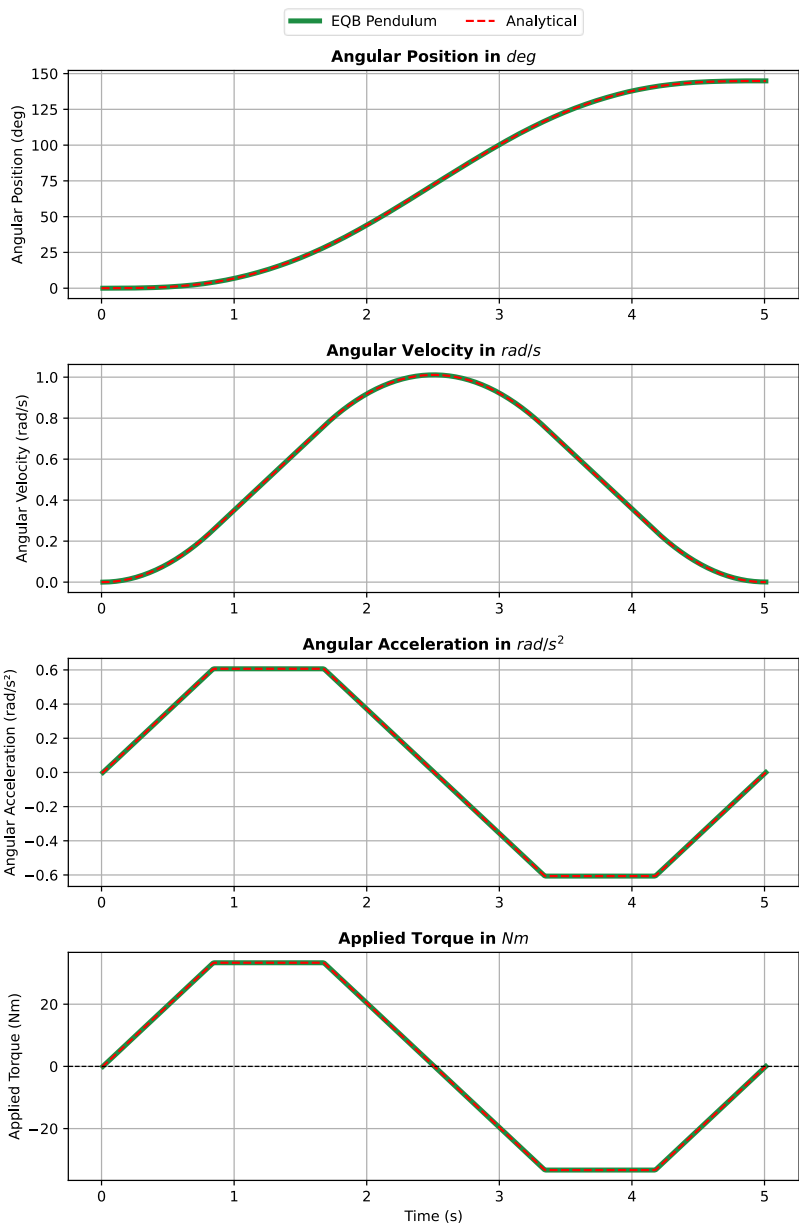
- **FEM:**
 - Internal representation via grid functions for displacement, velocity and acceleration;
 - Extracts lumped parameters from distributed fields using weighted projections
- **OpenSim:**
 - Coordinate query for value(=position), speed, and acceleration
- **FMU:**
 - internal variable state of non-fixed variables is obtained via getReal()

State Transfer: Injection / setState(state)

- **FEM:**
 - Rotates reference geometry by q , and computes velocity using radius and angular velocity
 - Angular acceleration is used to set acceleration grid function
 - Improvements:
 - Handover also previous values for q , ω , and α
 - Update torque parameter
 - Perform Newton Minimization step to step new displacement field
 - Update velocity and acceleration using Newmark-scheme
 - *(only valid for rigid body kinematics assumption with small elastic deformations)*
- **OpenSim:**
 - position and velocity are directly set for coordinate
 - actuator torque is overridden and realize method is called to apply new torque to joint
- **FMU:**
 - Reinitialization with new initial conditions required (FMU version does not allow to set FMU state)
 - Requires state adaption (q to q_0 , ω to ω_0)



6. MultiCompPendulum: Model Comparison



- Time dependent torque application and time dependent mode switching
- First switch and synchronization due to hysteresis (dwell_time = 0.05 s)
- States are consistently handed over for currently active to new CoSimComponent
- Larger error in the angular acceleration for equation-based **FMU** due to **reinitialization**
- Limitation: alpha cannot be set directly, but is computed by the applied torque**

Event 1: t = 0.0500 s OpenSim → FEM			
Variable	Retrieved	Current	Difference
q (rad)	1.09188e-05	1.09188e-05	-1.38913e-19
omega (rad/s)	0.000727921	0.000727921	-3.25261e-19
alpha (rad/s²)	0.036396	0.036396	1.38778e-17
torque (Nm)	2	2	-2.22045e-16

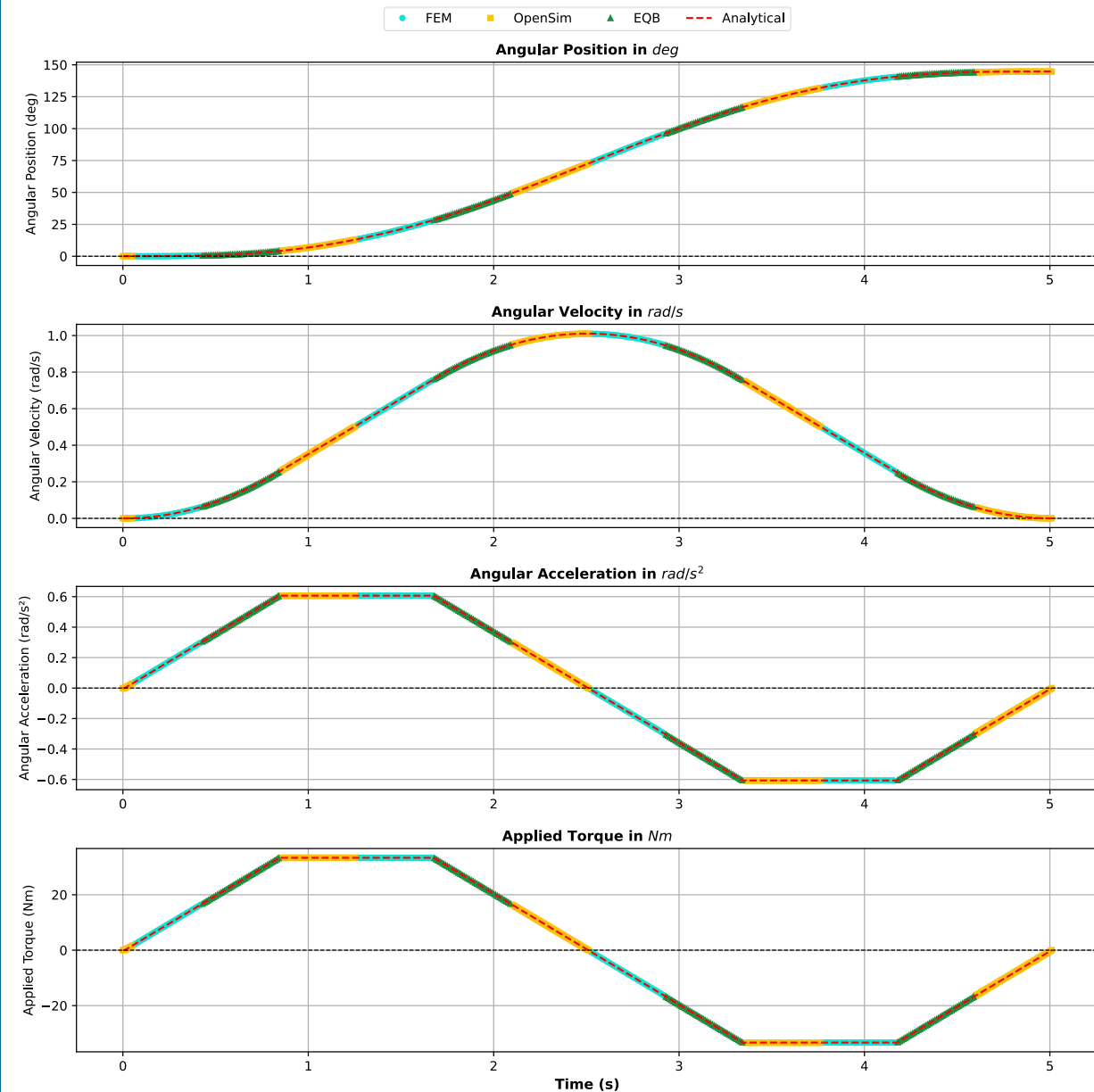
Event 2: t = 0.4200 s FEM → EQB			
Variable	Retrieved	Current	Difference
q (rad)	0.00843405	0.00843405	0
omega (rad/s)	0.0613637	0.0613637	0
alpha (rad/s²)	0.298448	0.305727	0.00727921
torque (Nm)	16.8	16.8	0

Event 3: t = 0.8400 s EQB → OpenSim			
Variable	Retrieved	Current	Difference
q (rad)	0.0698673	0.0698673	0
omega (rad/s)	0.252358	0.252358	0
alpha (rad/s²)	0.606601	0.606601	1.11022e-16
torque (Nm)	33.3333	33.3333	0

Event 4: t = 1.2500 s OpenSim → FEM			
Variable	Retrieved	Current	Difference
q (rad)	0.224319	0.224319	8.32667e-17
omega (rad/s)	0.501064	0.501064	-3.33067e-16
alpha (rad/s²)	0.606601	0.606601	0
torque (Nm)	33.3333	33.3333	7.10543e-15

Event 5: t = 1.6700 s FEM → EQB			
Variable	Retrieved	Current	Difference
q (rad)	0.488264	0.488264	0
omega (rad/s)	0.755827	0.755827	0
alpha (rad/s²)	0.606601	0.604174	-0.0024264
torque (Nm)	33.2	33.2	0

Event 6: t = 2.0900 s EQB → OpenSim			
Variable	Retrieved	Current	Difference
q (rad)	0.850393	0.850393	0
omega (rad/s)	0.947186	0.947186	0
alpha (rad/s²)	0.298448	0.298448	1.11022e-16
torque (Nm)	16.4	16.4	0



Framework Core Expansion:

- **New Components:**
 - OpenSimComponent: General Component Class for OpenSim models ✓
 - FEMComponent: General Component Class for netgen/NGSolve FEM models
- **Multicomponent Concept:**
 - Combine several representations of the same “thing” (e.g., pendulum FMU + OpenSim + FEM)
 - Implement state synchronization and rule-based switching between models ✓
- **Hybrid Capabilities:** ⌚
 - Support discrete and continuous states
 - Introduce hybrid DAE event handling (zero-crossing detection, event-triggered switching)
- **File-based (FEM) simulation tool integration concept** ⌚
 - Develop a workflow for file-based (FEM) model integration (openCFS, CalculiX, etc.)

User Interaction & System Assembly

- **Goal: Provide an intuitive, guided workflow inside Jupyter**
- **Multi-Step Wizard Interface (Standard & Expert Mode)**
 - Model selection -> FMU Export -> system assembly -> configuration -> simulation
 - Live preview of Modelica code ⌚
- **Configuration Management:**
 - Save and reload full system configurations (components, connections, parameters)
 - Implement configuration validation
- **Visualization:** ✓
 - Generate system graphs
 - Enable live plotting and simulation diagnostics for monitoring
- **Result Handling:** ⌚
 - Standardize simulation metadata and result exports for traceability

Simulation Engine & Computational Causality:

- **Goal: Enable robust, adaptive, and event-driven co-simulation**
- **Multi-rate / Adaptive Time Stepping:** ✓
 - Each component advances with its local solver time base
 - synchronized at communication points
- **Causality Solver:** ⌚
 - Extend execution order computation to handle cyclic dependencies
 - Algebraic loop resolution

Framework Integration & Validation

- **Goal: Consolidate core modules and ensure framework stability** ✓
- **Testing and Automation:**
 - Update and expand unit tests for all framework layers
 - Unit tests for core classes, wrappers, and port connections
 - Integration tests for system assembly and simulated run
 - Cross-platform validation (Windows/Linux/Mac)
- **Data Layer:** Structured result logging for reproducible simulation data
- **Documentation and Packaging:** ⌚
 - Refactor into modular pyproject-based package
 - Add tutorials, notebook, and licensing for distribution